

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL - 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	Shaik Ashraf
Register Number	192324088

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I Shaik Ashraf/192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ 

Assessment Tasks - Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced

through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

Sol:

A retail chain implements a cloud-based inventory system connecting POS terminals and dashboards to centralized cloud services via APIs, using cloud storage for product/transaction/analytics data and platform services for demand forecasting, secured through VPNs, firewalls, and API gateways.

1. Understanding of the Case - Key Requirements

- **Real-time synchronization:** Stock levels must update instantly across all store POS terminals and the central dashboard.
- **Multi-store scalability:** System must support hundreds of stores without performance degradation.
- **Data analytics & forecasting:** Historical sales/transaction data must feed demand-forecasting models.
- **Secure remote access:** Store devices connect over public networks and need protected, authenticated channels.
- **High availability:** Inventory checkout and billing cannot tolerate downtime during business hours.

2. Application of Cloud Concepts - Layered Architecture

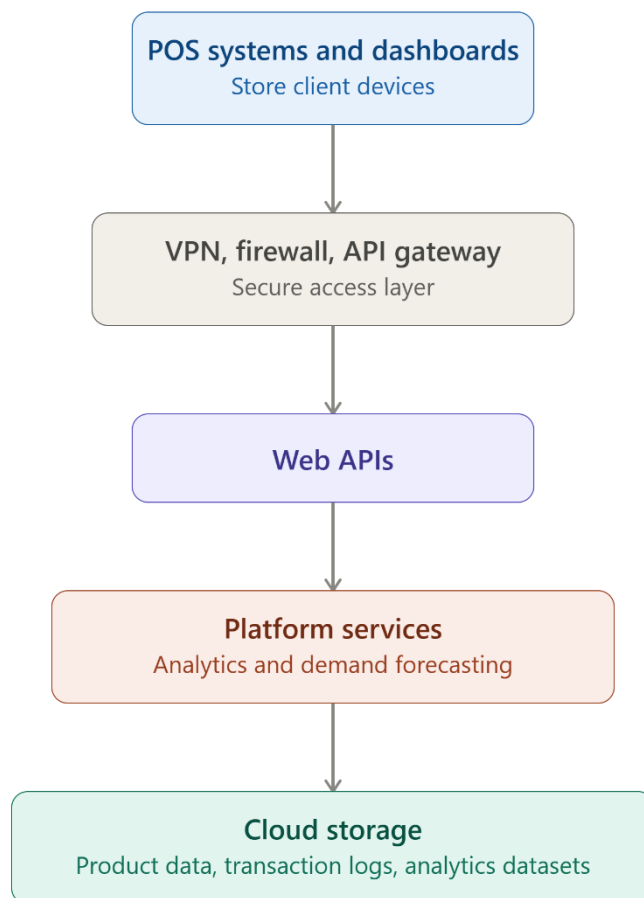
Layer	Component	Cloud Service / Technology
Client Layer	POS systems, admin dashboards	Web/thick clients calling REST APIs
API Layer	API Gateway	Amazon API Gateway / Azure API Management - routes, throttles, authenticates requests
Compute / Platform	Order & inventory microservices	PaaS (e.g., AWS Elastic Beanstalk / Azure App Service) or containers on Kubernetes
Storage	Product catalog, transaction logs	Relational DB (Amazon RDS) + Object storage (Amazon S3) for logs and reports
Analytics/PaaS	Demand forecasting	Managed ML/analytics service (AWS SageMaker, Azure Synapse) reading from a data lake
Network & Security	VPN, firewall, API gateway	Site-to-site VPN linking stores to cloud VPC; WAF and security groups control traffic

3. Solution Design

Proposed data flow:

- POS/dashboard clients send authenticated REST calls to the API Gateway.
- API Gateway validates tokens, applies rate limits, and routes to inventory/order microservices (PaaS/containers).
- Microservices read/write product and transaction data in a managed relational database; bulk logs go to object storage.
- A scheduled analytics pipeline extracts data into the platform's ML service to generate demand forecasts, feeding restocking dashboards.
- All store-to-cloud traffic is carried over VPN tunnels; firewalls and security groups restrict access to only required ports/services.

WORKFLOW:



Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a Google-Docs-like tool where browser clients perform real-time editing on documents stored in distributed cloud storage with version control, synchronized via Web APIs, with global low-latency delivery and strong identity/access security.

1. Understanding of the Case - Key Requirements

- **Real-time concurrent editing:** Multiple users must edit the same document simultaneously without conflicts.
- **Distributed, versioned storage:** Documents need geo-distributed storage with rollback/version history.
- **Cross-device synchronization:** Changes must sync instantly across browsers, desktop, and mobile clients.
- **Global low latency:** Users across regions need fast load/save times.
- **Identity, encryption & permissions:** Only authorized users should view/edit specific documents.

2. Application of Cloud Concepts - Layered Architecture

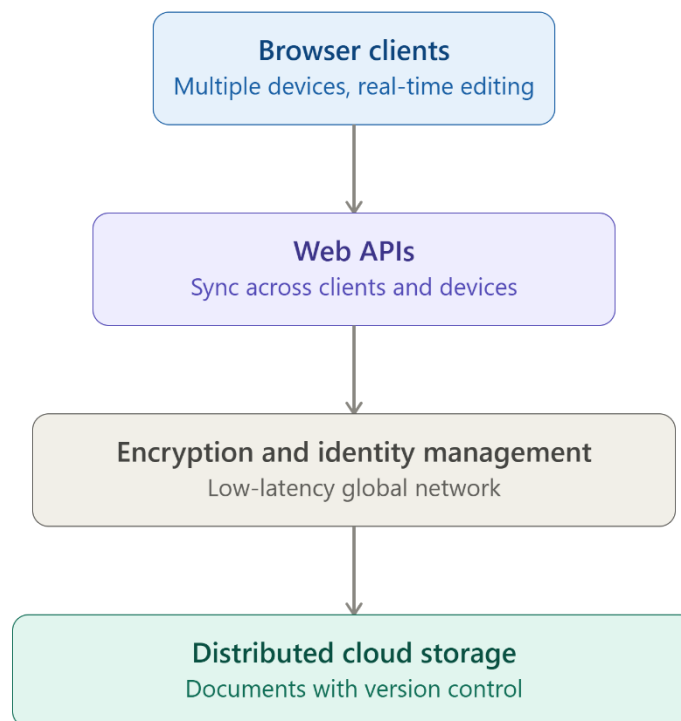
Layer	Component	Cloud Service / Technology
Client Layer	Browser editor (rich text, collaborative UI)	WebSocket + REST client using Operational Transform / CRDT libraries
API / Sync Layer	Real-time sync service	Web API + WebSocket gateway (e.g., API Gateway with WebSocket support) for change events
Compute / Platform	Collaboration & version-control service	PaaS microservices managing document state, merge, and history
Storage	Document content, revisions	Distributed object storage (e.g., Amazon S3 / Azure Blob with versioning enabled) + metadata DB
Content Delivery	Global low-latency access	CDN (CloudFront/Azure CDN) caching static assets close to users
Security	Auth, encryption, permissions	Identity provider (OAuth/SSO), TLS in transit, server-side encryption at rest, role-based sharing

3. Solution Design

- Browser clients open a WebSocket session through the API gateway for live change events, alongside REST calls for document open/save.
- A collaboration microservice merges concurrent edits using CRDT/OT and persists snapshots plus incremental diffs to versioned object storage.
- Static assets and recently accessed document snapshots are cached at CDN edge nodes to minimize global latency.
- Access is gated by an identity provider issuing short-lived tokens; each document carries role-based permissions (owner/editor/viewer) checked on every API call.
- Data is encrypted with TLS in transit and server-side encryption at rest; audit logs record every access and edit for compliance.

This architecture satisfies real-time collaboration, durability (versioned storage), global performance (CDN), and enterprise-grade security (IAM + encryption).

WORKFLOW:



Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system for web and mobile clients, communicating with backend services through RESTful Web APIs, using cloud storage for menu images/invoices/customer data, virtual machines and managed Kubernetes for scalable infrastructure, and token-based/RBAC security over HTTPS.

1. Understanding of the Case - Key Requirements

- **Multi-platform client support:** Web and mobile apps must consume the same backend consistently.
- **Scalable order processing:** Traffic spikes (meal times, promotions) require elastic scaling.
- **Secure media & data storage:** Menu images, invoices, and customer data need durable, secure storage.
- **Reliable, available service:** Order and payment flows must stay available with minimal downtime.
- **Strong authentication & authorization:** Customers, restaurants, and delivery agents need role-specific access.

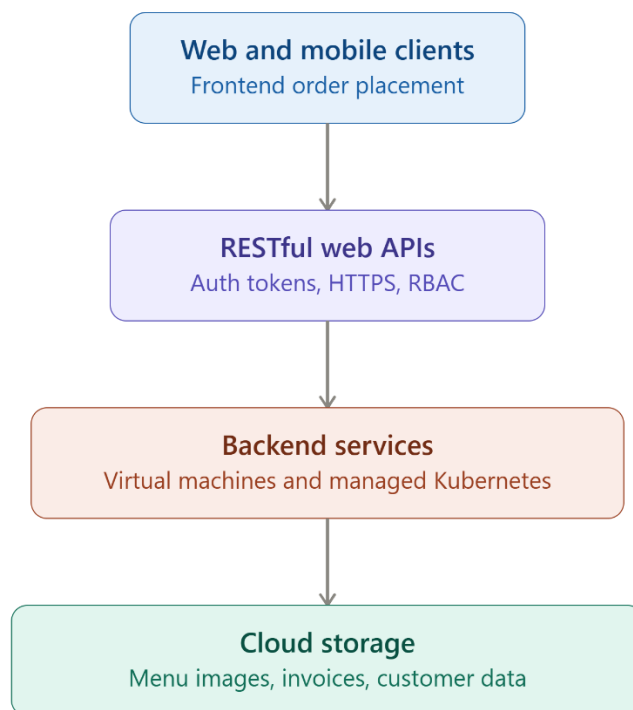
2. Application of Cloud Concepts - Layered Architecture

Layer	Component	Cloud Service / Technology
Client Layer	Web app, mobile app	Consume RESTful Web APIs over HTTPS
API Layer	REST API gateway	Central gateway issuing/validating authentication tokens (JWT/OAuth2)
Compute / Platform	Order, payment, delivery microservices	Containerized services on managed Kubernetes (e.g., Amazon EKS / Azure AKS) for scalability
Infrastructure	Backing VMs / node pools	Virtual machines as Kubernetes worker nodes with auto-scaling groups
Storage	Menu images, invoices, customer data	Object storage (S3/Blob) for images/invoices; managed DB (RDS/Cosmos DB) for structured customer/order data
Security	AuthN/AuthZ, transport security	Token-based auth (JWT), HTTPS everywhere, role-based access control (customer/restaurant/admin)

3. Solution Design

- Web/mobile clients call REST APIs over HTTPS; the API gateway validates JWT/OAuth2 tokens before forwarding requests.
- Order, payment, and delivery-tracking logic run as independently scalable microservices on a managed Kubernetes cluster backed by auto-scaling VM node pools.
- Menu images and invoices are stored in object storage; customer profiles and order records live in a managed database, both encrypted at rest.
- Role-based access control differentiates customers, restaurant partners, and administrators, each restricted to their permitted API operations.
- Horizontal pod autoscaling in Kubernetes handles meal-time traffic surges, while health checks and multi-zone deployment maintain availability.

WORKFLOW.



Cross-Case Summary Comparison

Aspect	Case Study 1 (Retail Inventory)	Case Study 2 (Doc Collaboration)	Case Study 3 (Food Ordering)
Primary Client	POS systems, dashboards	Browser (real-time editor)	Web + mobile apps
Communication	REST APIs via API gateway	REST + WebSocket for live sync	RESTful Web APIs over HTTPS
Storage Focus	Product, transaction, analytics data	Versioned documents, distributed storage	Menu images, invoices, customer data
Platform Service	Analytics / demand forecasting (PaaS)	Collaboration & versioning service	Managed Kubernetes for microservices